CIS Classifier

Custom CNN vs. Pretrained (MobileNetV2) Transfer Learning —
Model Comparison and Django Deployment

Project	CIS Image Classification (CIS vs. non-CIS)
Author	Asad
Course / Type	Final Year Project (FYP)
Deployment	Django web application

1. Introduction & Problem Statement

This project implements a binary image classification system that distinguishes CIS images from non-CIS images. The dataset consisted of 1,766 labeled images split across two classes (**CIS_train** and **non_CIS_train**), organized in a standard folder-per-class structure.

The project scope required: (1) a custom-built convolutional neural network trained from scratch, (2) a pretrained model fine-tuned via transfer learning, built in three tweaked variants to study the effect of different training strategies, (3) a quantitative comparison of all resulting models, and (4) a Django-based web application allowing a user to upload an image and view predictions and confidence scores from every trained model side by side.

2. Dataset & Preprocessing

The dataset was split 80/20 into training and validation sets using Keras's `image_dataset_from_directory`, yielding 1,413 training images and 353 validation images across the two classes. All images were resized to 224x224 pixels (the standard input size for MobileNetV2 and comparable pretrained architectures) and pixel values were normalized to the [0, 1] range.

For the custom model and one pretrained variant, data augmentation (random horizontal flip, rotation, and zoom) was applied during training to improve generalization on a comparatively small dataset.

Note on evaluation methodology: due to the project's time constraints, the validation split doubled as the test set used for final model evaluation, rather than a fully separate held-out test set. This is disclosed here as a limitation of the evaluation methodology (see Section 6).

3. Custom Model Architecture & Training

A convolutional neural network was designed and trained from scratch to serve as a baseline that does not benefit from any pretrained knowledge. The architecture consists of four convolution + max-pooling blocks (32, 64, 128, and 128 filters respectively), followed by a flatten layer, a dropout layer (rate 0.5) for regularization, a dense layer of 128 units, and a final sigmoid output unit for binary classification. Data augmentation was applied prior to the convolutional blocks.

The model was compiled with the Adam optimizer and binary cross-entropy loss, and trained for 10 epochs.

Metric	Value
Final Training Accuracy	~96.3% (train), see note
Final Validation Accuracy	62.9%
Behavior	Noisy convergence; accuracy still trending upward at epoch 10

Note: reported 'final training accuracy' figures for this run tracked closely with validation accuracy through most epochs (both in the 0.60-0.65 range), consistent with a model that had not yet converged within the 10-epoch budget, rather than one that had overfit.

The custom model's relatively low validation accuracy compared to the pretrained models (Section 5) illustrates the core value of transfer learning: a small dataset of ~1,400 training images is often insufficient to train a deep CNN from scratch to high accuracy within a short training budget.

4. Pretrained Model & Three Variants

MobileNetV2, pretrained on ImageNet, was selected as the transfer-learning backbone for its strong balance of accuracy and computational efficiency, making it well suited to a tight project timeline and eventual deployment in a web application. Three variants were trained to test different transfer-learning strategies:

- **Version 1 — Frozen Base (baseline transfer learning):** the MobileNetV2 convolutional base was frozen entirely; only a new classification head (global average pooling, dropout 0.3, dense 128, sigmoid output) was trained. This tests how well off-the-shelf ImageNet features transfer without any adaptation.
- **Version 2 — Fine-Tuned (deeper adaptation):** the last 30 layers of the MobileNetV2 base were unfrozen and trained jointly with the classification head, using a lower learning rate (1e-5) to avoid destroying pretrained weights. This tests whether adapting higher-level features to the CIS domain improves results.
- **Version 3 — Augmented + Regularized:** the base was kept fully frozen (as in V1), but data augmentation was added before the base model and dropout was increased to 0.5. This tests whether stronger regularization narrows the train/validation gap seen in V1.

All three variants were trained for 10 epochs on the same training/validation split as the custom model, for direct comparability.

5. Results & Model Comparison

All four trained models (custom CNN, and the three pretrained variants) were evaluated on the same held-out data using accuracy, precision, recall, and F1 score (weighted average), along with confusion matrices.

Model	Accuracy	Precision	Recall	F1 Score
Custom CNN	62.89%	62.36%	62.89%	62.06%
Pretrained V1 (Frozen)	92.07%	92.26%	92.07%	92.01%
Pretrained V2 (Fine-Tuned)	83.85%	83.84%	83.85%	83.85%
Pretrained V3 (Augmented)	84.14%	84.20%	84.14%	84.16%

Discussion

Pretrained V1 (frozen base) achieved the best overall performance (92.07% F1), substantially outperforming both the custom CNN and the other two pretrained variants. This confirms that ImageNet-pretrained features, even without any fine-tuning, generalize well to this classification task given the limited dataset size. V1 did, however, show a noticeable gap between training accuracy (96.3%) and validation accuracy (92.1%), a mild indicator of overfitting.

Pretrained V2 (fine-tuned) underperformed relative to V1 (83.85% F1). Its training curve was still improving at epoch 10, suggesting the lower learning rate required for safe fine-tuning needed more training epochs than the project timeline allowed to fully converge.

Pretrained V3 (augmented + higher dropout) scored close to V2 (84.14% F1) but showed the smallest train/validation gap of the three pretrained variants, indicating the added regularization achieved its intended effect of reducing overfitting, at some cost to peak accuracy.

Custom CNN trailed all pretrained variants substantially, underscoring that training a deep network from scratch on ~1,400 images within a 10-epoch budget is insufficient to match transfer learning.

6. Limitations & Out-of-Distribution Behavior

Test set methodology: as noted in Section 2, the validation split was reused as the test set for final evaluation rather than using a fully independent held-out set. Reported metrics should therefore be interpreted as in-distribution performance estimates rather than a fully unbiased generalization measure.

Out-of-distribution inputs: informal testing through the deployed web application with images unrelated to the training domain (e.g., casual photographs of people in everyday settings) revealed that most models — including the strongest performer, V1 — defaulted to predicting the CIS class with moderate-to-high confidence, rather than expressing uncertainty. This is expected behavior for a standard binary classifier: the architecture has no mechanism to reject inputs that fall outside the training distribution and must always output a class probability. This finding does not contradict the validation results in Section 5, which

measure performance specifically on data drawn from the training distribution; rather, it highlights a known limitation of binary classifiers on out-of-distribution data and would require either an explicit 'unknown/other' class or a dedicated out-of-distribution detection mechanism to address.

7. Web Application (Django Deployment)

All four trained models were deployed behind a single Django web application. On application startup, all four `.h5` model files are loaded once into memory (via a custom `AppConfig.ready()` hook) rather than per-request, keeping prediction latency low. The user-facing flow is:

- User uploads an image through a simple HTML form.
- The image is preprocessed identically to training (resized to 224x224, normalized to [0,1]).
- The preprocessed image is passed through all four loaded models.
- Each model's predicted class, confidence score, and known test-set accuracy (from Section 5) are displayed together in a results table.

This design allows a single uploaded image to be evaluated against every trained model simultaneously, directly supporting the project's comparison goal within a live, interactive tool.

8. Conclusion & Future Work

This project demonstrated a clear, quantified advantage of transfer learning over training a custom CNN from scratch on a modestly sized image dataset: the best pretrained variant (V1, frozen base) reached 92.07% F1 versus 62.06% F1 for the custom model. Among the three pretrained variants, simple frozen-feature transfer learning outperformed both deeper fine-tuning and heavier regularization within the available training budget, though each variant illustrated a different, understandable trade-off.

With more time, the following improvements would be prioritized:

- Expanding and diversifying the dataset, particularly the non-CIS class, to reduce reliance on background/context shortcuts.
- Training Version 2 (fine-tuned) for substantially more epochs to allow proper convergence at its lower learning rate.
- Constructing a fully independent, held-out test set separate from the validation split used during training.
- Adding out-of-distribution detection or an explicit 'unknown' class to make the deployed system more robust to unrelated inputs.

Overall, the results support a practical takeaway for similarly constrained projects: when time and data are limited, pretrained transfer learning is the more reliable choice over training a custom architecture from scratch.